

Bugfix Log

Revision: 20 **Last modified:** 2026-06-16T23:55:00Z

Per CONST-MD-Bugfix-Documentation, every bug surfaced during implementation gets a permanent entry below: title, root cause, affected files, fix description, regression guard.

Entries are append-only; do not edit historical entries except to add clarification footnotes.

2026-04-27 — boba-jackett implementation (plan 2026-04-27-jackett-management-ui-and-system-db.md)

1. Master-key two-step write silent data-loss window

Severity: HIGH (silent credential loss possible on crash mid-bootstrap).

Root cause: Initial implementation of `bootstrap.EnsureMasterKey` generated `BOBA_MASTER_KEY` in memory, returned it to the caller, and *then* wrote `.env` in a separate step. If the process crashed, was killed, or the host lost power between those two operations, the caller would proceed to encrypt credentials with a key that disk had no record of — making them permanently undecipherable.

Affected files: - `qBittorrent-go/internal/bootstrap/bootstrap.go` - `qBittorrent-go/internal/envfile/write.go`

Fix: Collapse the generate + write into a single `envfile.Atomic write (tmpfile + fsync + rename)` before the function returns. Caller never sees a key that hasn't been durably persisted. Commit `c0f20bc`.

Regression guard: - `qBittorrent-go/internal/bootstrap/bootstrap_test.go::TestEnsureMasterKeyDoesNotDuplicateHeader` - `qBittorrent-go/internal/envfile/write_test.go` atomic-write test layer

2. SQLite database file mode 0644 (world-readable credentials)

Severity: CRITICAL (any local user could read encrypted credentials + ciphertext-only attack surface).

Root cause: modernc.org/sqlite creates database files with the process's default umask, which on the container yielded 0644 (world-readable). The Go service's .env was already 0600 via envfile.Atomic, but boba.db was leaking — a defense-in-depth breach since the rows are encrypted, but still a violation of the principle “the DB file should be readable only by the service owner”.

Affected files: - qBitTorrent-go/internal/db/conn.go

Fix: After the first Open() succeeds, db.Open calls os.Chmod(path, 0600) (and the -wal / -shm siblings if present), collapsing the file mode to owner-only. Commit 8405167.

Regression guard: - Layer 4 security challenge challenges/scripts/boba_db_file_perms_challenge.sh - qBitTorrent-go/internal/db/credential_leak_test.go

3. AutoconfigResult nil slices marshalled as JSON null

Severity: MEDIUM (silent client-breaking schema drift; UI rendered “undefined” badges instead of “0 items”).

Root cause: Go's json.Marshal serializes a nil slice as null, not []. When Autoconfigure() ran on a fresh DB with no discovered credentials, the discovered_credentials, configured_now, and already_present fields all marshalled as null. The OpenAPI 3.1 schema declared them as array, so clients (including the Angular dashboard) treated null as a contract violation.

Affected files: - qBitTorrent-go/internal/jackett/autoconfig.go

Fix: Pre-allocate empty slices ([]string{}) at the top of Autoconfigure() so a “nothing to do” run still serializes as {..., "discovered": [], "configured_now": [], ...}. Commit 6f3dbaf.

Regression guard: - Layer 6 contract test qBitTorrent-go/tests/contract/openapi_test.go (validates each named field is array per OpenAPI schema even on empty runs).

4. Catalog ReplaceAll empty-input wipe risk

Severity: HIGH (would permanently wipe the indexer catalog if Jackett returned an empty response; UI would show no indexers and fuzzy matcher would have nothing to match against).

Root cause: repos.IndexerCatalog.ReplaceAll(rows []Row) unconditionally DELETED existing rows then INSERTED the new ones. If rows was empty (Jackett momentarily unhealthy, transient 502, or catalog parse failure), the delete still happened, leaving an empty catalog table. Next refresh would have nothing to compare against.

Affected files: - qBitTorrent-go/internal/db/catalog.go (or equivalent repo file)

Fix: Refuse the operation early with errors.`New("repos: ReplaceAll refusing empty replacement")`. The caller (autoconfig orchestrator) treats this as a soft error, logs it, and leaves the previous catalog intact. Commit 8d71df1.

Regression guard: - `qBittorrent-go/internal/jackettapi/catalog_test.go::TestRefreshCatalogAllTemplatesFailReturns200WithErrors`

5. `TestSearchHandler_QueueFull` pre-existing flake (NOT a boba-jackett bug)

Severity: LOW (test-only; flagged for follow-up).

Status: NOT FIXED in this plan. Confirmed unrelated to boba-jackett work.

Location: `qBittorrent-go/internal/api/api_test.go`

Symptom: Occasional intermittent failure under load when the search queue fills before the test's wait returns.

Action: Logged here for traceability so future audits don't mis-attribute it. Open issue suggestion: stabilise by injecting a deterministic queue-full hook instead of racing against real goroutine scheduling.

2026-06-09 — Session 7 fixes (`env_loader`, `AsyncMock`, `gamestorrents`)

6. `env_loader` flaky test: `KEY2` leak across test ordering

Severity: LOW (test-only flake; not a production bug).

Root cause: `test_comment_lines_ignored` in `tests/unit/test_env_loader.py` asserted `os.environ.get("KEY2")` is `None` after calling `load_env_files()` with a config file where `KEY2=commented_out` was commented out. However, `load_env_files` has a “first wins” policy — if `KEY2` was already present in `os.environ` from a prior test's `load_env_files` call, the function would not override it. Under `pytest-randomly` ordering, the test that sets `KEY2` sometimes ran before the comment-line test, causing intermittent failure.

Affected files: - `tests/unit/test_env_loader.py`

Fix: Added explicit `os.environ.pop("KEY1", None)` and `os.environ.pop("KEY2", None)` at the START of the test (before calling `load_env_files`), ensuring a clean env state regardless of test execution order. The finally block was retained as a safety net.

Regression guard: - `tests/unit/test_env_loader.py::test_comment_lines_ignored` — ran 2147/2147 twice consecutively under random ordering with zero failures.

7. AsyncMock warning in search deep-coverage tests

Severity: LOW (test warning; not a production bug).

Root cause: `test_iporrents_login_no_cookies` in `tests/unit/merge_service/test_search_deep_coverage.py` used `AsyncMock()` for `login_resp` and `mock_session`. These objects are used as context managers (`async with session.post(...) as resp`), not as directly awaited coroutines. `AsyncMock.__aenter__` returns a coroutine, but the test's mock setup didn't properly wire `__aenter__`/`__aexit__`, causing "coroutine was never awaited" `RuntimeWarning`.

Affected files: - `tests/unit/merge_service/test_search_deep_coverage.py`

Fix: Changed `AsyncMock()` to `MagicMock()` with explicit `__aenter__` and `__aexit__` stubs, matching the actual usage pattern (context manager, not coroutine).

Regression guard: - `tests/unit/merge_service/test_search_deep_coverage.py` — 0 `AsyncMock` warnings from this test (was 3).

8. gamestorrents _parse_size B-substring bug (documented, not fixed)

Severity: MEDIUM (plugin returns 0 for all realistic file sizes).

Root cause: Same class of bug as BOB-013 (torrentkitty). The multipliers dict in `_parse_size` iterates in insertion order: `{"B": 1, "KB": 1024, ...}`. Since "B" is a substring of "KB", "MB", "GB", and "TB", the check if unit in `size_str` matches "B" first for all realistic sizes. Then `size_str.replace("B", "")` leaves a trailing unit character (e.g. "35.2 G") which `float()` cannot parse, returning 0.

Affected files: - `plugins/gamestorrents.py` (line 74-86)

Fix: NOT FIXED in this session. Documented via tests that assert actual behavior (tests suffixed `_b_substring_bug`). Fix requires reordering the dict keys to check longest units first (same approach as BOB-013).

Regression guard: - `tests/unit/test_plugin_gamestorrents.py::TestParseSize` — 8 tests documenting the bug across GB/MB/KB/TB/commas/uppercase.

2026-06-09 — Session 8 parallel plugin tests + gamestorrents fix

9. gamestorrents _parse_size B-substring bug (fixed)

Severity: MEDIUM (plugin returned 0 for all realistic file sizes).

Root cause: Same class of bug as BOB-013 (torrentkitty). The multipliers dict in `_parse_size` iterated in insertion order: `{"B": 1, "KB": 1024, ...}`. Since "B" is a substring of "KB", "MB", "GB", and "TB", the check if unit in `size_str` matched "B" first for all realistic sizes. Then `size_str.replace("B", "")` left a trailing unit character (e.g. "35.2 G") which `float()` could not parse, returning 0.

Affected files: - `plugins/gamestorrents.py` (line 74-86)

Fix: Reordered dict keys to check longest units first: `{"TB": ..., "GB": ..., "MB": ..., "KB": ..., "B": 1}`. Same approach as BOB-013.

Regression guard: - `tests/unit/test_plugin_gamestorrents.py::TestParseSize` — 8 tests all pass with correct byte values for GB/MB/KB/TB/B/comma/uppercase.

10. piratebay `import os` placed after use (documented, not fixed)

Severity: LOW (only affects non-magnet torrent file downloads, which are rare).

Root cause: `piratebay.py:175` has `import os` placed after `os.fdopen` on line 172. When downloading a non-magnet torrent file, `os.fdopen` is called before `os` is imported, causing `UnboundLocalError`.

Affected files: - `plugins/piratebay.py` (line 172-175)

Fix: NOT FIXED in this session. Documented via test `test_torrent_file_download_known_bug_os_import_order`. Fix requires moving `import os` to the top of the `download_torrent` method or module level.

Regression guard: - `tests/unit/test_plugin_piratebay.py::TestDownloadTorrent::test_torrent_file_download_known_bug_os_import_order`

11. `torlock.search()` does not catch exceptions from `retrieve_url()`

Severity: LOW (network errors crash the entire search loop instead of failing gracefully).

Root cause: `torlock.py search()` calls `retrieve_url()` without a try/except, so a network error propagates up and crashes the search loop.

Affected files: - `plugins/torlock.py`

Fix: NOT FIXED in this session. Documented via test `test_search_exception_propagates`. Fix requires wrapping `retrieve_url()` in try/except with graceful error handling.

Regression guard: - `tests/unit/test_plugin_torlock.py::TestSearch::test_search_exception_propagates`

2026-06-09 — Session 8 wave 2: nyaa/kickass/anilibra/torrentgalaxy/yts

12. nyaa download_torrent missing import re (documented, not fixed)

Severity: MEDIUM (any nyaa.si URL passed to download_torrent raises NameError).

Root cause: plugins/nyaa.py:170 calls re.search() but import re is absent from the module. The search() method works because re is imported transitively via other modules, but download_torrent fails with NameError: name 're' is not defined.

Affected files: - plugins/nyaa.py (line 170)

Fix: NOT FIXED in this session. Documented via tests that assert the NameError. Fix requires adding import re at the top of the file.

Regression guard: - tests/unit/test_plugin_nyaa.py::TestDownloadTorrent — 6 tests including the NameError documentation.

13. kickass comma-separated size not matched by regex (documented, not fixed)

Severity: LOW (sizes like 1,234.5 MB silently parse to 0).

Root cause: The HTMLParser regex \d+\.\d+ doesn't match comma-separated numbers like 1,234.5 MB. The comma is not handled in the regex or the replacement logic.

Affected files: - plugins/kickass.py

Fix: NOT FIXED in this session. Documented via test test_comma_in_size_not_matched_by_regex. Fix requires updating the regex to handle commas.

Regression guard: - tests/unit/test_plugin_kickass.py::TestHTMLParserFeed::test_comma_in_size_not_matched_by_regex

14. kickass crash-prone patterns (fixed)

Severity: MEDIUM (empty responses from retrieve_url crash the plugin).

Root cause: Three methods in kickass.py called retrieve_url() without try/except or empty-response guards: __retrieve_download_link(), download_torrent(), and search(). When retrieve_url returned None or empty string, subsequent re.search/re.sub calls crashed with TypeError.

Affected files: - `plugins/kickass.py` (lines 61, 74, 94)

Fix: Added try/except + empty-response guards to all three methods. Each now handles empty/None responses gracefully (returns “NotFound”, prints fallback URL, or breaks the loop respectively).

Regression guard: - `tests/unit/test_plugin_kickass_guards.py` — 13 tests proving empty response, None response, `ConnectionError`, `TimeoutError`, and malformed HTML don’t crash.

2026-06-09 — Waves 5-8: B-substring epidemic across 9 community plugins

15. Systemic `_parse_size` B-substring bug (9 instances found, all fixed)

Severity: MEDIUM (all 9 plugins returned 0 for KB/MB/GB/TB — silently broken size reporting).

Root cause: The `_parse_size` method in each plugin uses a `multipliers` dict iterated in insertion order. With keys ordered `{"B": 1, "KB": 1024, ...}`, the check “B” in `size_str` matches the “B” substring inside “KB”, “MB”, “GB”, and “TB”. Then `size_str.replace("B", "")` leaves a trailing character (e.g. “1.5 G”) which `float()` cannot parse, returning 0.

Affected plugins (9 total): | Plugin | Status | |———|———| | `gamestorrents.py` | Fixed (BOB-024) | | `megapeer.py` | Fixed (BOB-042) | | `one337x.py` | Fixed (BOB-043) | | `extratorrent.py` | Fixed (BOB-044) | | `torrentfunk.py` | Fixed (BOB-045) | | `therarbg.py` | Fixed (BOB-047) | | `pctorrent.py` | Fixed (BOB-052, by subagent) | | `bitru.py` | Fixed (BOB-054) | | `xbsub.py` | Fixed (BOB-057) | | `yihua.py` | Fixed (BOB-058) |

Fix: Reorder dict keys longest-unit-first: `{"TB": ..., "GB": ..., "MB": ..., "KB": ..., "B": 1}`. Same approach as BOB-013 (torrentkitty).

Global regression guard: 10 dedicated test files, each with explicit `_parse_size` assertions for all 5 units (B/KB/MB/GB/TB).

16. Systematic `import re` omissions (3 instances found, all fixed)

Root cause: Three plugins called `re.search()` or `re.compile()` without importing `re` at module level. The `search()` method worked via transitive imports from helper modules, but `download_torrent()` failed with `NameError`.

Affected plugins: | Plugin | Status | |———|———| | `nyaa.py` | Fixed (BOB-035) | | `audiobookbay.py` | Fixed (BOB-042) | | (torlock `search()` also lacks exception handling — documented BOB-029) |

Fix: Added `import re` at module level.

17. bt4g infinite loop (test-only, fixed)

Severity: LOW (test hang, not production).

Root cause: `search()` has a `while True` loop that breaks when `parser.noTorrents` is `True`. Tests that used `return_value=MATCHING_HTML` (not `side_effect`) caused every page to match, triggering infinite loop.

Fix: Changed to `side_effect=[MATCHING_HTML, EMPTY_HTML]` so page 2 triggers the `noTorrents` break condition.

2026-06-13 — BobaLink extension: flaky-perf-test hardening + WCAG contrast fixes

Batch verified together by `extension/ci-ext.sh` → **CI-EXT: PASS**, full suite **632 passed (632)**, `tsc --noEmit` clean, `npm run lint` 0/0, chrome+firefox builds loadable, §11.4.38 asset-verify pass, both store zips ≥ 10 KiB. Every fix below carries a permanent regression guard (§11.4.135) that was RED before the fix.

18. Junk-flood DoS security test — load-coupled absolute-wall-clock FAIL-bluff

Severity: MEDIUM (flaky test — §11.4.1 FAIL-bluff; blocked the green gate).

Root cause (FACT, not guess): `extension/tests/security/scanner-hostile-input.test.ts` asserted a tight absolute `expect(elapsed).toBeLessThan(5000)` on a 50k-anchor orchestrator scan. On a shared/oversubscribed host (load-avg 15) the *correct* scan measured **6608 ms**; in isolation it passes < 5000 ms (proven 5/5). Product behaviour was always correct (exactly 2 magnets detected, all junk excluded) — only the absolute timing tripped.

Affected files: `extension/tests/security/scanner-hostile-input.test.ts`.

Fix: replaced the tight 5000 ms budget with a generous **30 s hang-ceiling** (catches a true hang / quadratic explosion — a real $O(n^2)$ over 50k anchors takes minutes) and moved the rigorous machine-independent DoS guard to a new perf test. Correctness assertions retained.

Regression guard: new `extension/tests/perf/orchestrator-scaling.perf.test.ts` asserts a metamorphic sub-quadratic scaling ratio $(t(10 \cdot N)/t(N) < 30)$; linear ≈ 10 , quadratic ≈ 100 — dimensionless, immune to host load) with a golden-good/golden-bad oracle self-validation (§11.4.107(10)): `linearRatio=9.9` accepted, `quadraticRatio=113` rejected.

19. crypto.perf p99 budget — contention-coupled FAIL-bluff

Severity: MEDIUM (flaky test).

Root cause (FACT): `extension/tests/perf/crypto.perf.test.ts` asserted the 80 ms budget on `dist.p99.PBKDF2 × 2` is CPU-bound; under concurrent-suite contention p99 spiked to **164 ms** while intrinsic cost stayed ~23–41 ms (proven in isolation). The newly-added heavy parallel `perf/stress` suites added the contention that exposed it.

Affected files: `extension/tests/perf/crypto.perf.test.ts`.

Fix: assert the budget on `dist.min` (intrinsic, contention-robust — contention only adds time). A real $\geq 3.6\times$ algorithmic regression still inflates the min past 80 ms; correctness (EXACT plaintext recovery) is asserted every round.

Regression guard: the same test (now contention-robust); isolated min ~23 ms $\leq$ 80; full-suite CI-EXT: PASS.

20. `parsers.perf` scaling-ratio — contention-coupled FAIL-bluff

Severity: MEDIUM (flaky test).

Root cause (FACT): `extension/tests/perf/parsers.perf.test.ts` computed the per-link scaling ratio from the **median** time at each size. The small (1000-link, ~5 ms) median is noise-sensitive under contention; the ratio tripped **3.09** > 3 while the intrinsic ratio is ~1.0 (proven 1.34 / 1.00 in isolation).

Affected files: `extension/tests/perf/parsers.perf.test.ts`.

Fix: compute per-link cost from the **min** run at each size (intrinsic). A genuine $O(n^2)$ regression still shows ratio ≈ 5 in the min run, so the cap of 3 keeps full regression-catching power while never flaking.

Regression guard: the same test; isolated ratio $1.24 \leq 3$; full-suite CI-EXT: PASS.

21. Three real WCAG 2.1 AA colour-contrast defects (popup theme tokens)

Severity: MEDIUM (accessibility — WCAG SC 1.4.3, real product defect).

Root cause (computed, §11.4.6): three popup theme tokens fell below the 4.5:1 normal-text contrast floor. Found by the new `extension/tests/a11y/focus-and-contrast.a11y.test.ts` (computes contrast ratios from the REAL committed CSS; analyzer self-validated against WCAG reference extremes), kept RED until fixed.

Affected files: `extension/src/popup/styles.css` (each token in both the `@media prefers-color-scheme` block and the explicit `[data-theme]` class).

Fix (same hue family, minimal visual change, $\geq 4.6:1$): - `--text-faint dark` on `#1a1a2e`: `#7a7a90` (4.07) $\rightarrow$ `#838399` (**4.61**) - `--text-faint light` on `#f5f6fa`: `#8a8a9c` (3.14) $\rightarrow$ `#6c6c7e` (**4.76**) - `--warning-text light` on `#fff6e0`: `#9a6a00` (4.40) $\rightarrow$ `#946400` (**4.78**)

Regression guard: the 3 a11y tests flipped RED $\rightarrow$ GREEN on the fix (proving they catch the defect) and are permanent §11.4.135 guards.

2026-06-13 — BobaLink extension wave-11: options-page WCAG a11y fixes

Found by the new extension/tests/a11y/options-contrast-motion.a11y.test.ts (computes contrast ratios from the REAL committed src/options/styles.css; analyzer self-validated against WCAG reference extremes), kept RED until fixed. Batch verified by extension/ci-ext.sh → **CI-EXT: PASS** with the wave-11 coverage additions. All RED→GREEN proven; the §11.4.120 reconciliation rewrote the gates to assert the NEW mechanism (not fake-passed, not reverted).

22. Save-button text below WCAG AA over its gradient (SC 1.4.3)

Severity: MEDIUM (accessibility, real). **Root cause (computed):** .btn-primary has color:#fff over linear-gradient(--accent → --accent-2); white on the lighter --accent (#667eea) end is only **3.66:1** (14px normal-weight text needs ≥ 4.5). **Fix:** the global --accent is ALSO .nav-item.active text on a dark sidebar where darkening it would REDUCE contrast, so the button uses a **button-local** darker indigo start #5a6fce (**4.57:1**); the --accent-2 end is already 6.37:1. **Affected:** src/options/styles.css .btn-primary. **Guard:** the test now reads the button's ACTUAL gradient stops and asserts #fff ≥ 4.5 over every stop.

23 & 24. Field labels + nav-hover invisible in light theme — hard-coded literals (SC 1.4.3)

Severity: MEDIUM (accessibility, real — labels effectively unreadable in light mode). **Root cause (computed):** .field > label (#d0d0e0) and .nav-item:hover (#e0e0e0) were hard-coded **dark-theme** literals that never re-resolved per theme → **1.40:1** and **1.12:1** on the light backgrounds. **Fix:** tokenized to theme-aware tokens — label → var(--text) (10.6/13:1), nav-hover → var(--text-strong) (15.5/16.1:1) — legible in both themes. **Affected:** src/options/styles.css. **Guard:** the §11.4.120-reconciled test asserts the selectors now use a var(--token) (not a literal) AND the resolved token clears AA in BOTH themes.

25 & 26. No prefers-reduced-motion escape (SC 2.3.3 / 2.2.2)

Severity: MEDIUM (accessibility, real). **Root cause:** the options CSS ships @keyframes fadeIn (panel reveal on tab activation) + transitions, and the popup CSS ships transitions, but neither had a @media (prefers-reduced-motion: reduce) block — motion-sensitive users had no way to suppress it. **Fix:** added the standard reduced-motion block (zeroing animation/transition durations) to BOTH src/options/styles.css and src/popup/styles.css. **Guard:** the a11y tests assert each stylesheet ships non-trivial motion AND a reduced-motion block (RED→GREEN on the fix).

2026-06-13 — BobaLink extension wave-12: scaling-ratio FAIL-bluff hardening

Batch verified by `extension/ci-ext.sh` → **CI-EXT: PASS**, full suite **799 passed (799)** (+38 over wave-11: crypto-tamper 17, link-scanner-coverage 10, highlight-manager 11 — all green, no product defects). The one fix below was a flaky test exposed by the heavier 799-test load.

27. infohash dedup scaling-ratio test — single-run + tiny-floor FAIL-bluff (§11.4.50)

Severity: MEDIUM (flaky test — §11.4.1 FAIL-bluff; blocked the green gate).
Root cause (FACT): `tests/security/infohash-detection-hostile.test.ts`'s relative-scaling guard used a `SINGLE-run tLarge / Math.max(tSmall, 0.05)`; on a sub-ms baseline the 0.05 ms floor made the ratio noise-dominated (**53.6** observed under full-suite contention vs ~ 4 intrinsic). Worse, the ≤ 40 threshold was too loose to even catch the quadratic it guards ($4\times$ input → linear ≈ 4 , quadratic ≈ 16 — both < 40). Passed at 761 tests, tripped at 799. **Fix:** measure the MIN over 7 reps at each size (intrinsic, contention-robust — host stalls only ADD time) and tighten the threshold to **10** (sits BETWEEN linear ≈ 4 and quadratic ≈ 16 , so it now genuinely catches an $O(n^2)$ regression while never flaking). Verified stable 5/5 in isolation + full-suite **CI-EXT: PASS**. **Affected:** `extension/tests/security/infohash-detection-hostile.test.ts`. **Note:** the sibling stress test's median estimator was initially assessed robust — but it ALSO flaked one wave later under heavier load (see entry 28), confirming that median is not enough; min is.

2026-06-13 — BobaLink extension wave-13: tab-group scaling median→min + working-tree hygiene

Batch verified by `extension/ci-ext.sh` → **CI-EXT: PASS**, full suite **814 passed (814)** (+15 over wave-12: popup-states 5 — genuine partial-Send-All gap; plus two prior-session test files brought into git — `options-save-flow 4` + `offline-queue-persistence 6` — that were sitting UNTRACKED in the working tree). No product defects.

28. tab-group scaling-ratio test — median estimator spikes under load (§11.4.50)

Severity: MEDIUM (flaky test — §11.4.1 FAIL-bluff; blocked the green gate).
Root cause (FACT): `tests/stress/orchestrator-ratelimiter-tabgroup.stress.test.ts`'s scaling guard used the MEDIAN of 5 reps; under the heaviest concurrent load (814 tests) several reps landed on a busy core and dragged the median up, so the $10\times$ -input ratio spiked past the 25 ceiling while the intrinsic ratio is $\sim 5\text{--}8$. (This refutes the wave-12 assessment that median was robust — entry 27's note.) **Fix:** switched the shared timing helper from median to **MIN** over reps (the minimum is the intrinsic-cost estimator — host stalls only ADD time, so the fastest run is closest to true cost; a genuine $O(n^2)$ regression still inflates every run

including the min) and bumped reps (orchestrator 3→5, tab-group 5→9). Verified stable 3/3 in isolation (ratios 4.76/6.41/8.25 vs the 25 ceiling) + full-suite CI-EXT: PASS. **Affected:** extension/tests/stress/orchestrator-ratelimiter-tabgroup.stress.test.ts. **Lesson (§11.4.118):** the robust estimator for CPU-bound scaling-ratio tests under concurrent load is MIN-of-reps, not median/mean — all six flaky-scaling fixes this session converge on it.

2026-06-13 — BobaLink extension wave-14: live-7187 harness hardening

Live-harness prep to de-risk the operator-gated live round-trip. The default suite is unchanged at **814 passed (814)** (CI-EXT: PASS); the live test + challenge are out-of-suite and operator-run.

29. live_detect_send_challenge — §11.4.14 cleanup-race leaves an orphan synthetic torrent

Severity: MEDIUM (anti-bluff/hygiene defect in a live Challenge). **Root cause (FACT):** challenges/extension/live_detect_send_challenge.sh cleaned up the synthetic torrent it adds ONLY when its secondary qBittorrent torrents/info probe reported present. But the proxy adds the magnet itself; that separate probe can race/auth-fail and read absent while the magnet IS added — so a PASS could leave an orphan synthetic torrent in qBittorrent (a §11.4.14 quiescence violation). **Fix:** cleanup now keys off the **proxy's authoritative per-url verdict** (results[0].status == "added") OR the probe, re-logs-in for a fresh SID when needed, and warns honestly if delete-login fails (never reds the run at teardown). Also fixed a /bin/sh -n parse issue (apostrophe/metachar in added comments — §11.4.67). **Affected:** challenges/extension/live_detect_send_challenge.sh. **Verification:** clean SKIP (exit 77) when the backend is down (no fail-open), bash -n + sh -n PASS, mutation-verified (no-op/partial response stubs → FAIL); the cleanup trigger confirmed to fire on a simulated proxy-added + probe-absent response (the old code would not). The up-path runs when the operator brings the stack up.

2026-06-13 — Boba broader-project (merge-service / Go / plugins / frontend) verification + 2 fixes

A full-project verification pass (parallel subagents, all run against the REAL toolchains, anti-bluff captured evidence) confirmed the broader Boba backend is green: **Go qBitTorrent-go 14 packages PASS under -race, vet clean, no data races; plugins: 60 .py compile, 12 curated contract-valid, 901 plugin tests pass** (under .venv/py3.13); **Angular frontend: build GREEN + 342 unit tests pass; merge-service Python: 4149 unit tests pass, ruff clean** (under .venv/py3.13). Two genuine items found:

30. ci.sh / scripts/run-tests.sh false-red under a host python3 < 3.12

Severity: MEDIUM (the project’s own manual CI gate false-reds — a §11.4.1 FAIL-bluff at the toolchain layer). **Root cause (FACT, independently found by two subagents):** both scripts invoke a **bare python3**, which resolves to the host default. pyproject.toml declares requires-python = ">=3.12" and the suite uses tomllib + PEP-604 X | Y union syntax that a python3 < 3.12 cannot even COLLECT. On this host python3 = **3.9.25**, so python3 -m pytest tests/unit/ aborts: **“Interrupted: 36 errors during collection”** (TypeError: unsupported operand), while the project .venv (py3.13) collects **4149** and passes all. **Fix:** both scripts now select a >=3.12 interpreter via a _select_python helper (prefers \$PYTHON → .venv/bin/python → python3.13 → python3.12 → a bare python3 only if it is ≥3.12), aborting with a clear message if none qualifies. Backward-compatible: a host already on python3 ≥ 3.12 selects the same interpreter. **Affected:** ci.sh (lines for py_compile + the unit/integration/e2e pytest steps), scripts/run-tests.sh (the 5 exec ... pytest modes). **Verification (captured):** bash -n clean both; the selector picks .venv py3.13; bare py3.9 → “36 errors during collection” vs selected interpreter → “4149 tests collected” on the same tests/unit/ dir.

31. Go backend — 2 uncovered critical paths closed (coverage, not a defect)

Type: Task (coverage). The qBitTorrent-go merge_search RunSearch goroutine-orchestration loop (every prior test passed a nil client) and the sse_broker concurrent pub/sub had zero real coverage. Added 4 anti-bluff tests (internal/service/coverage_test.go, real httptest qBittorrent server, no mocks) — RunSearch completes/accumulates + StartFails→failed + ContextCancel→abort, and an 8-publisher/16-subscriber SSEBroker churn under -race. **Anti-bluff proof:** a no-op return nil stub in RunSearch made all three RunSearch tests FAIL, then reverted (production clean, §11.4.84). Verified GREEN under -race (service package ok 5.933s, no data race). Production code unchanged.

2026-06-13 — Boba merge-service hermetic specialized suites: 5 test-quality fixes (product correct)

A verification pass over the merge-service’s hermetic specialized suites (stress, chaos, property, contract, concurrency, memory, observability, security — run under .venv/py3.13, the suites BG-1’s tests/unit pass did not cover) found the PRODUCT is correct + secure (bandit 0 HIGH; CORS hardened; no product defect) but surfaced 5 real TEST-quality defects across the anti-bluff classes. All test-side only — download-proxy/src/ unchanged; each verified green under .venv.

32. Five merge-service test-quality defects (§11.4.1 / §11.4.3 / §11.4.50 / §11.4.120)

- **tests/stress/test_pipeline_stress.py** — §11.4.50 **flaky absolute-wall-clock threshold**. Gated on `max(latencies_ms) < 2000` (absolute); a lone GC/scheduler stall spiked one iteration to ~140 ms (~25× the 5.6 ms median) — an absolute bound is the wrong oracle. Fixed to a RELATIVE bound `max ≤ 200 × max(p50, 1.0 ms)` (an $O(n^2)$ blowup inflates the median too, so it still fires; a lone outlier no longer flakes). Verified: 3 passed.
 - **tests/contract/test_crossapp_theme_contract.py** — §11.4.1/§11.4.3 **FAIL-instead-of-SKIP**. The 3 `@requires_compose` tests' own docstring promises “skip cleanly when the proxy is unreachable”, but `_fetch()` did a bare `urlopen` → `URLError` → **FAIL** when down. Fixed `_fetch()` to catch only `CONNECTION` errors → `pytest.skip` (with `./start.sh -p reason`); `HTTPError` still returns the real status and `AssertionError` is NOT swallowed (a real missing-bridge still **FAILs** when the proxy is up). Verified: 3 **SKIP** when down, suite 27 passed when collectable.
 - **tests/security/test_cors.py** — §11.4.120 **stale gate asserting REMOVED insecure behavior**. `TestCORSWildcardDefault` asserted `Access-Control-Allow-Origin == "*" as the default`, but the product was deliberately hardened to a secure-by-default localhost/dev allowlist (wildcard removed, CONTINUATION known-issue #5). The TEST lagged the security improvement. Reconciled to assert the NEW secure default — a known localhost origin reflected EXACTLY, an unknown LAN origin NOT reflected and never *. Anti-bluff: a wildcard-revert fails the `!= "*" assertions (RED-on-old/GREEN-on-new)`, not a tautology. Verified.
 - **tests/observability/test_metrics_exist.py** — §11.4.3 **infra-FAIL instead of SKIP**. A `@requires_compose` test made an unconditional `httpx.get(:7187/metrics)` → `ConnectError` **FAIL** when down. Added the existing `@merge_service_required` skip guard → **SKIPS** cleanly. Verified.
 - **tests/security/test_jackett_autoconfig_secrets.py** — **toolchain-path FAIL**. `subprocess.run(["bandit"])` raised `FileNotFoundError` (bandit is in `.venv/bin`, not on bare `PATH`). Added a `_resolve_bandit()` helper (prefers the running interpreter's bin dir, then `shutil.which`, **SKIP** if genuinely absent). Product clean: `.venv/bin/bandit` on `jackett_autoconfig.py` → 0 **HIGH** findings. Verified.
-

2026-06-13 — Boba broken-doc-links + perf-test container-boot (the last test-type suites)

Ran the remaining test types (performance/benchmark/load/docs) under `.venv`. Benchmark: 9 infra-free pass (500×–4760× threshold headroom — not flaky, correctly left unchanged) + 5 by-design error-not-skip infra-down (intentional). Load: only a Locust `locustfile.py` (no `pytest` tests). Two real items fixed (BUGFIXES 33, 34).

33. docs/ test caught 8 broken internal markdown links + 1 test FAIL-bluff

Severity: LOW-MEDIUM (broken doc links + a §11.4.1 test FAIL-bluff). **Root cause + fix (FACT):** tests/docs/test_no_broken_links.py (resolves links relative to each doc's own dir, the markdown convention) flagged 8 docs. Seven were genuine broken links — docs/*_Summary.md linked to siblings with a spurious docs/ prefix ([Architecture Summary](docs/Architecture_Summary.md) in a doc already inside docs/ → resolves to docs/docs/...). Fixed by dropping the prefix ([](docs/X) →](X)); docs/Scripts_Summary.md's](scripts/pre_build_verification.sh) →](.. /scripts/...) (the script is at repo-root). The 8th — a research doc — was a **TEST FALL-BLUFF**: the link extractor matched [\${tab.title}](\${tab.url}) inside a **JS code block** (not a doc link). Fixed the extractor to STRIP fenced + inline code before matching. Anti-bluff verified (§1.1): a real broken link in **prose** still FAILs (the strip only removes code, not real links — not a tautology). **Affected:** 7 docs/*_Summary.md (+regenerated .html/.pdf), tests/docs/test_no_broken_links.py. **Verified:** tests/docs → 129 passed / 0 failed / 2 honest skip (mkdocs absent).

34. Performance test BOOTED CONTAINERS + hung 120 s on infra-down (§11.4.3 / no-boot)

Severity: MEDIUM (a test that actively boots infra + hangs). **Root cause (FACT):** tests/performance/test_concurrent_search.py::TestConcurrentSearch consumed a merge_service_live fixture that, on ports-down, ran podman/docker compose up -d then _wait_for_port(timeout=120) — pytest-timeout killed the wait → 7 setup-ERRORs + 2 min wall-clock, and it was ACTIVELY booting containers. The suite's siblings (stress/security/observability) all use the fast merge_service_required probe-and-SKIP guard. **Fix:** applied that same merge_service_required guard (1 s socket+/health probe, never boots containers) → 7 honest infra-down SKIPs. Non-tautology: when the stack is up the class still runs every real assertion (status 200, latency ceilings, p50/p95) unchanged. (The benchmark suite's separate error-not-skip design is INTENTIONAL and was left untouched per §11.4.122/§11.4.6.) **Affected:** tests/performance/test_concurrent_search.py. **Verified:** 7 SKIP cleanly, no container boot, download-proxy/src/ unchanged.

2026-06-13 — start.sh boot completely broken: boba-ctl up -d flag mismatch

35. ./start.sh aborted EVERY boot at the container-start step (-d flag)

Severity: HIGH (the documented startup path ./start.sh did not work at all). **Root cause (FACT, found by actually running the boot):** start.sh::start_container() runs the generic \$COMPOSE_CMD up -d. In the default boba-ctl mode \$COMPOSE_CMD = scripts/boba-ctl.sh, whose up|down case did exec "\$BOBA_CTL_BIN" "\$@" — passing the compose-style -d (detach)

straight to the `cmd/boba-ctl/boba-ctl` Go binary, whose `up` defines only `-profile/-wait`. Result every boot: flag provided but not defined: `-d` → `[ERROR] Failed to start container → exit 1`, before any container was created. **Fix:** `scripts/boba-ctl.sh`'s `up|down` case now strips `-d/--detach` (`boba-ctl` runs detached by default) so it is a true drop-in for `compose up -d`. **Affected:** `scripts/boba-ctl.sh`. **Verified:** `bash -n` clean; the re-run boot proceeded past the step into `podman-compose ... up -d` (created `pod_boba`, pulled all images).

Operator-environment note (NOT a project defect, reported per §11.4.6): with the `-d` bug fixed, the boot then HUNG inside `podman-compose` itself — it created `pod_boba` + pulled every image, then froze at 0.0% CPU (`STAT=SN`) before creating any container. `podman` itself works (`podman run alpine` → OK), so the hang is specific to `podman-compose`, which on this host runs under homebrew `python@3.14` (a likely-incompatible combo; `podman-compose` support for 3.14 is not established). Native `podman compose` just delegates to the same `podman-compose`. This is an operator-environment toolchain issue (fix: run `podman-compose` under a supported python, e.g. 3.11/3.12, or repair the `podman-compose` install), surfaced to the operator — the project's `./start.sh` orchestrator code is now correct.

Clarifying footnote (2026-06-13, root-caused — supersedes the python@3.14 hypothesis above): the hang was NOT `python@3.14`. Captured live: the `vfkit` VM shares are `/Users`, `/private`, `/var/folders` and `podman machine inspect` → `Mounts: []` — the repo lives on `/Volumes/T7` (external SSD), which is NOT shared into the `podman applehv` VM, so the first `podman create` stalls on an unreachable bind path. Proven by both `python 3.13` AND `3.14` hanging identically (refuting the version theory). Fix (operator-authorised, §11.4.122): recreated the machine with `--volume /Volumes/T7:/Volumes/T7 --cpus 4 --memory 6144`; all 5 containers then booted healthy. On macOS the `network_mode: host` ports live in the VM, so an SSH `-L` tunnel (or `scripts/ensure-macos-tunnel.sh`) bridges `:7186/:7187` to the Mac.

2026-06-13 — qBittorrent 5.x API-compat: download round-trip broken (login + add)

Surfaced by the live `detect→send→torrent` round-trip (`extension/tests/live/download-endpoint.live.test.ts`) once the stack was finally booted (`qBittorrent v5.2.1`, `linuxserver/qbittorrent:latest`). Three real PRODUCT defects + two test/challenge-quality items, each proven with captured live evidence. The whole class: the proxy + tests spoke legacy qBittorrent's plain-text protocol (`200 "Ok."`) while the image speaks 5.x's JSON/204.

36. Proxy reported `auth_failed` for EVERY download — modern qBittorrent 204 login

Severity: CRITICAL (no download could reach qBittorrent). **Type:** Bug. **Root cause (FACT, captured live):** `download-proxy/src/api/routes.py` judged WebUI login with `resp.status == 200` and `body == "Ok."`. Modern qBittorrent (4.6+/5.x) returns 204 No Content with an EMPTY body + the `QBT_SID` cookie (`HTTP/1.1 204 / set-cookie: QBT_SID_7185=...`) → `status != 200` AND `body != "Ok."` → `{"status":"auth_failed"}` even

though login succeeded. The defect lived at FOUR login call-sites (/download, /download/upload, /downloads/active, /auth/qbittorrent) — the active-downloads dashboard widget + credential-save endpoint were broken too. **Fix:** new `_qbit_login_succeeded(status, body, cookies)` — authoritative version-independent signal is “the server issued a QBT_SID/QBT_SID_<port> session cookie” (exact/prefix match, not a loose "SID" substring), legacy Ok. body as fallback; all four call-sites migrated. **Affected:** download-proxy/src/api/routes.py. **Regression guard (§11.4.135):** tests/unit/test_qbit_login_compat.py incl. test_reproduces_pre_fix_defect_old_check_rejected_modern_204 (§11.4.115 polarity). **Verified live:** POST /api/v1/auth/qbittorrent admin/admin → {"status":"authenticated","version":"v5.2.1"}; the round-trip went from auth_failed-SKIP to GREEN.

37. Proxy reported failed for successful adds — modern qBittorrent JSON add + duplicate 409

Severity: CRITICAL (a torrent that actually landed was reported failed). **Type:** Bug. **Root cause (FACT, captured live):** the add-success check was `body.lower().startswith("ok")`. Modern qBittorrent /api/v2/torrents/add returns a JSON summary, not Ok.: {"added_torrent_ids": ["<hash>"], "failure_count":0, "pending_count":0, "success_count":1} — the torrent IS added (confirmed in /torrents/info) but the proxy saw { → failed with the JSON as detail. SECOND, related: BobaClient retries on network/timeout; attempt 1 added the magnet but was slow → client timed out → retry re-sent the SAME infohash → qBittorrent 409 Conflict (duplicate) → failed (proven: one round-trip = TWO /api/v1/download requests in the proxy log). **Fix:** new `_qbit_add_succeeded(status, body)` — accepts the modern JSON (added_torrent_ids non-empty OR success_count/pending_count >= 1, with defensive int-coercion so a malformed "N/A" count classifies as failure, never crashes the add path), legacy Ok., AND treats 409 Conflict as idempotent SUCCESS (the torrent IS present — makes a retried add safe); all three add call-sites migrated. **Affected:** download-proxy/src/api/routes.py. **Regression guard:** tests/unit/test_qbit_login_compat.py (modern-JSON / pending-only / all-failed / 409-duplicate / non-numeric-count / legacy-Ok / Fails / 415 cases). **Verified live (×3, §11.4.50):** the live round-trip + challenges/extension/live_detect_send_challenge.sh both GREEN — the synthetic infohash is INDEPENDENTLY confirmed present in qBittorrent’s real /torrents/info, then cleaned up (§11.4.14).

38. Live test + challenge could not cross-check modern qBittorrent (test-quality, no product defect)

Severity: MEDIUM (anti-bluff held: tests SKIP’d honestly, never false-PASS’d). **Type:** Task. **Root cause:** (a) the live test’s own confirm-login helper had the same legacy status==200 && "Ok." assumption → could not read /torrents/info on 204 → honest SKIP. (b) the challenge cross-checked qBittorrent at :7185 (container-internal, not reachable off-host on macOS). **Fix:** (a) confirm-login now accepts 200/204 + keys on the Set-Cookie SID (mirrors the proxy); (b) challenge default qBittorrent base :7185 → :7186 (the documented WebUI access port, reachable on Linux + macOS). **Affected:** extension/tests/live/download-

endpoint.live.test.ts, challenges/extension/
live_detect_send_challenge.sh. **Verified:** both GREEN with real infohash-
present confirmation + §11.4.14 cleanup.

Code review (§11.4.142/§11.4.134): an independent adversarial review caught 2
BLOCKING (2 of the 4 login sites initially missed; an `int("N/A")` crash path) + 1
warning (loose "SID" substring) — all remediated + guarded; the re-review returned
a clean GO.

2026-06-13 — merge dashboard: search-result buttons dead during search + results collapse on completion

Operator-reported (searched “Linux”, clicked buttons): “Magnet/Download/qBit
buttons don’t work while the search is running; after completion they work but the
results are just a fraction of what we saw streaming.” Root-caused via systematic-
debugging (§11.4.102), fixed TDD (RED proven, §11.4.115), independent code
review GO (§11.4.142/§11.4.134), live-verified on the running stack.

39. Search-result buttons (Magnet / Download / qBit) were a NO- OP while a search was in progress

Severity: HIGH (core user action unusable mid-search). **Type:** Bug. **Root cause**
(FACT, code evidence): the Angular dashboard grid renders `*ngFor="let r of
sortedResults(); let i = index"` and the buttons call `doMagnet(i)/
doSchedule(i)/ doDownload(i)`, but the handlers read `this.results()[i]`.
`results()` is EMPTY during a search (only `loadSearchResults()` sets it on
`search_complete`) and in a DIFFERENT order after — while the grid shows
`sortedResults()` (= `liveResults()` during search). So mid-search `results()`
`[i]` is undefined → `if (!r) return` → every button did nothing; after search the
index desynced from the sorted display (wrong-row risk). **Fix:** handlers resolve the
row from `this.sortedResults()[index]` — the SAME array the template
iterates. Plus the deferred qBit-login callback now captures the ROW object
(`executeScheduleRow(r)`) instead of re-resolving by index, eliminating a search-
completes- mid-dialog wrong-row race (review nit). **Affected:** `frontend/src/app/
components/dashboard/dashboard.component.ts`. **Regression guard**
(§11.4.115/§11.4.135): `dashboard.component.spec.ts` — 4 new tests (each
button fires for the live row mid-search + the deferred-login row-capture); RED-
proven (pre-fix → “expected one matching request, found none”). **Verified live:**
`challenges/extension/search_buttons_live_challenge.sh` PASS — all 3
button backends (`/api/v1/magnet/`, `/api/v1/download/file`, `/api/v1/
download`) satisfy their real contract; the synthetic magnet is added +
INDEPENDENTLY confirmed present in qBittorrent + cleaned up (§11.4.14). Full
frontend suite 345 green.

40. Search results collapsed to a fraction (50) on completion despite thousands streamed

Severity: HIGH (perceived data loss). **Type:** Bug. **Root cause (FACT, code + runtime evidence):** `get_search` (the endpoint the grid swaps to on `search_complete`) hardcoded for `m` in `merged[:50]`: (not even the requested limit). Captured live: a “Linux” search streamed **2153 results** into the live grid, but the final list was capped at **50** — the operator’s “just a fraction.” **Fix:** `get_search(search_id, req, limit: int = 0)` returns the FULL merged set by default (`display = merged` if `limit <= 0` else `merged[: max(0, limit)]`); the grid uses `cdk` virtual-scroll so the full set renders cheaply. **Affected:** `download-proxy/src/api/routes.py`. **Regression guard (§11.4.115/§11.4.135):** `tests/unit/test_search_results_not_capped.py` (3 tests: 73 merged → all 73 not 50; optional `?limit` truncates; sub-50 returned whole) — RED-proven (`cap-50 → 50≠73`). **Verified live (§11.4.123):** post-fix “Linux” search → completed total=1363 merged=862 RESULTS_LEN=862 (was 50). The final set now matches the merged count.

41. pytest tore down the operator’s running stack on every suite that touched docker_services

Severity: MEDIUM (test-infra; repeatedly killed the live stack mid-session). **Type:** Bug. **Root cause (FACT):** `tests/conftest.py` overrode `docker_setup` → `[]` (skip `compose-up`) but NOT `docker_cleanup`, so `pytest-docker`’s default teardown ran `compose down`, destroying a stack the suite did NOT bring up. Any suite using the `docker_services` fixture (e.g. the contract/security live-probing tests) nuked `qbittorrent` + the proxy on teardown. **Fix:** add a `docker_cleanup` fixture returning `[]` (mirrors `docker_setup`) — the suite is now a pure observer; startup/shutdown is owned by `./start.sh`. **Affected:** `tests/conftest.py`. **Composes §11.4.119** (single-resource-owner — tests must not seize a resource in use).

2026-06-13 — merge dashboard over a LAN IP: qBit login-loop + Jackett CORS (+ DNS-rebinding hardening)

Operator-reported while testing the dashboard via `http://192.168.0.132:7187` (a LAN IP, not localhost). Systematic-debugging (§11.4.102), TDD (RED-proven), independent + automated-security review, live-verified.

42. qBit button prompted for a qBittorrent login on EVERY click + never remembered it

Severity: HIGH (the qBit “send to qBittorrent” button was unusable). **Type:** Bug. **Root cause (FACT, code + runtime):** `auth.py::all_trackers_auth_status` computed `qbit_has_session = resp.status == 200` and `login_text.strip() == "Ok."` — the SAME legacy-qBittorrent defect fixed in `routes._qbit_login_succeeded` but in `auth.py`, which the earlier fix MISSED (§11.4.118 — fixed the reported instance, not the whole class). Modern qBittorrent (v5.2.1) returns `204` + the `QBT_SID` cookie, so `has_session` was ALWAYS false →

qbitAuthenticated() false → the dashboard opened the login dialog on every qBit-button click. SECOND: the probe ran only if creds:, so with no saved creds the default admin/admin was never even tried. **Fix:** always probe with the effective creds (saved else default admin/admin via QBITTORRENT_USER/PASS) and accept 204 + QBT_SID (mirrors_qbit_login_succeeded). **Affected:** download-proxy/src/api/auth.py. **Verified live:** /api/v1/auth/status → qbittorrent has_session=true → the dashboard auto-authenticates with the default creds, the qBit button sends directly without prompting.

43. Jackett button “0 Unknown Error” over a LAN IP — CORS (+ DNS-rebinding security hardening)

Severity: HIGH (Jackett page unusable off-localhost). **Type:** Bug. + **Security (HIGH).** **Root cause (FACT):** boba-jackett’s CORS defaultAllowedOrigins were hardcoded localhost/127.0.0.1, so the dashboard origin http://192.168.0.132:7187 was not allowed → the browser blocked the cross-origin call → status-0 “0 Unknown Error”. **Fix:** a sameHost rule allows an Origin co-hosted with the request — deriving the allowed origin FROM THE REQUEST (CLAUDE.md anti-bluff: no hardcoded localhost CORS). **Security (DNS rebinding, caught by automated security review):** an initial name-equality sameHost was vulnerable — an attacker DNS-rebinding evil.example→victim-LAN-IP gets a matching name in Origin+Host → CORS granted → response leaked. **Hardened:** sameHost matches ONLY IP-LITERAL hosts (net.ParseIP on both, net.SplitHostPort for IPv6) — a DNS-rebinding NAME can never match, while genuine LAN-IP access works. **Affected:** qBitTorrent-go/internal/jackettapi/cors_middleware.go. **Regression guards (§11.4.115):** cors_middleware_test.go — same-host-IP allowed, different-host blocked, **DNS-rebinding-name blocked** (RED-proven: the vulnerable version leaked ACA0=http://attacker.example), IPv6-literal allowed. **Verified live:** LAN-IP origin → ACAO echoed (page loads); attacker.example name origin → no ACAO (blocked). boba-jackett Go image rebuilt. **Independent review GO; automated security review: finding resolved.**

2026-06-14 — search results: large streaming list suddenly overwritten by a much smaller one

Operator-reported: “during the search... suddenly different much smaller results list comes and overwrites everything.” Systematic-debugging (§11.4.102) + operator product-decision (§11.4.66) + 2 parallel implementer subagents + 2 review rounds (§11.4.142/§11.4.134).

44. De-duplicated result list collapses on completion (now streamed progressively)

Severity: HIGH (looked like the search lost most results). **Type:** Bug. **Root cause (FACT, measured live):** during a search the dashboard streamed every per-tracker copy (liveResults, deduped only by tracker|link|name → ~1785 rows for “matrix”); on search_complete it swapped to the backend’s content-deduplicated merged set (get_search, ~663) → the count dropped ~2.3×. The dedup is **correct + intentional** (the same torrent mirrored on 51/37/31 trackers merges into one row — NO distinct torrent is lost); the jarring drop was purely the raw-live vs merged-

final view inconsistency. Operator chose (§11.4.66): “show the de-duplicated list as it builds.” **Fix:** the backend now streams the de-duplicated set PROGRESSIVELY via a new merged_update SSE event (re-merge accumulated results, throttled $\geq 1.5s$, + an unconditional FINAL emit before search_complete); serialization is a shared _serialize_merged_rows helper used by BOTH the stream AND get_search, and the FINAL emit reads the orchestrator’s authoritative cached merge (_last_merged_results) so it is byte-identical to get_search → ZERO drop at completion. The frontend renders the merged snapshot (liveMergedResults) progressively while searching, so the list builds smoothly to the unique count. **Affected:** download-proxy/src/api/streaming.py (_build_merged_update, merged_update emits), download-proxy/src/api/routes.py (_serialize_merged_rows), frontend/src/app/components/dashboard/dashboard.component.ts. **Regression guards (§11.4.115):** tests/unit/merge_service/test_merged_update_streaming.py (progressive emits; final == cached-merge CONTENT identity via a late-tracker distinguisher — RED-proven) + dashboard.component.spec.ts (merged_update renders merged; **no-shrink-on-completion guard** 50→400→663; completion fallback). **Verified live:** “matrix” → merged_update progression [0, 54, 68, 153, 289, 345], **final merged_update 345 == get_search 345** (zero drop). Full frontend suite 349 pass; backend 4 pass. **Review (§11.4.142/§11.4.134):** first pass NOT-GO (final-emit/get_search race + missing anti-bluff coverage) → both fixed + re-verified → clean GO.

2026-06-14 — merge dashboard result-row buttons: malformed multi-xt magnet, multi/no-add on qBit, no click feedback

Operator-reported on the merged search-results grid: “Magnet button fails with error, magnet link does not get generated!”; “qBit button does not work at all, press on it does not send torrent to qBitTorrent at all”; and “responsiveness of each clicked search results button must be flashing fast and with indicators (UI/UX) of something being prepared or processed!”. All three root-caused via live browser reproduction on 2026-06-14 (§11.4.107 anti-bluff). The first two share one structural cause: a merged search-results row aggregates many DISTINCT tracker-copies of one content item (each a different infohash), and both endpoints treated all of those source infohashes as if they were one torrent.

45. Magnet button generated a malformed multi-xt magnet (up to 21 xt=urn:btih: params)

Severity: HIGH (the Magnet button errored / produced an unusable magnet). **Type:** Bug. **Root cause (FACT, captured live):** a merged search-results row aggregates many DISTINCT tracker-copies of one content item, each carrying its own infohash. generate_magnet in download-proxy/src/api/routes.py joined EVERY source infohash into the magnet’s xt (xt = "&".join(...)), producing a magnet with up to 21 distinct xt=urn:btih: params. Confirmed live: an Ubuntu merged row’s Magnet dialog showed **21** distinct xt hashes. A magnet must identify exactly ONE torrent; qBittorrent rejects a multi-xt magnet → “magnet not generated”. **Fix:** build the magnet’s single xt from the PRIMARY (first = best / highest-seeded) source only, while STILL aggregating tracker (tr=) entries from all sources (preserving peer reach without ambiguous identity). **Affected:** download-proxy/

src/api/routes.py (generate_magnet); mirrored in the Go backend qBittorrent-go/internal/api/download.go (MagnetHandler). **Regression guard (§11.4.115/§11.4.135):** RED→GREEN tests/unit/test_download_merged.py::TestMergedMagnetGeneration::test_magnet_endpoint_builds_single_xt_from_primary + ::test_magnet_single_xt_for_21_source_merged_row; reconciled the stale gate tests/unit/api_layer/test_routes_coverage.py::TestGenerateMagnetMultiHash::test_multiple_btih_hashes per §11.4.120 (rewritten to assert the NEW single-xt mechanism, not fake-passed/reverted). **Verified live:** served-endpoint check (xt count == 1); the SAME Ubuntu merged row that streamed 21 xt hashes now produces exactly 1 — screenshots qa-buttons-multixt-magnet-BEFORE.png / qa-buttons-singlxt-magnet-AFTER.png at repo root.

46. qBit button added multiple wrong torrents (or none of the intended one) for a merged row

Severity: HIGH (the qBit “send to qBittorrent” button did not deliver the picked torrent). **Type:** Bug. **Root cause (FACT, captured live):** initiate_download in download-proxy/src/api/routes.py looped for url in request.download_urls[:5] and added EACH url. For a merged row (many distinct tracker-copies of one item) this added up to 5 DIFFERENT torrents instead of the single one the user picked. Confirmed live: a merged row added 3 of 5 distinct torrents — so the user saw multiple unintended torrents (or, when the intended copy failed first, not the one they meant). **Fix:** add the PRIMARY (first) source and STOP after the first successful add; fall through to the next source ONLY if a prior one fails (primary-with-fallback), so exactly one torrent — the intended one — lands. **Affected:** download-proxy/src/api/routes.py (initiate_download). **Regression guard (§11.4.115/§11.4.135):** RED→GREEN tests/unit/test_download_merged.py::TestMergedQbitAdd::test_qbit_adds_only_primary_source_of_merged_row (asserts exactly ONE /torrents/add call + added_count == 1; FAILED pre-fix with 3 adds).

47. Result-row buttons gave no processing feedback on click (no UI/UX busy state)

Severity: MEDIUM (UX — clicks looked unresponsive while the request was in flight). **Type:** Bug. **Root cause (FACT):** the Magnet / qBit / Download handlers fired their async work with no per-button busy state, so a click produced no visible “preparing / processing” indication while the request was in flight — the operator perceived the button as dead. **Fix:** a per-row, per-action busy signal in frontend/src/app/components/dashboard/dashboard.component.ts (isBusy(index, action)), wired into all three handlers (set on click, cleared on success AND on error). The template frontend/src/app/components/dashboard/dashboard.component.html adds aria-busy, a .busy class, and an inline ; dashboard.component.scss adds an instant :active press-flash, a .busy pulse animation, and the spinner — so every click flashes immediately and shows in-flight progress. **Affected:** frontend/src/app/components/dashboard/dashboard.component.ts, dashboard.component.html, dashboard.component.scss. **Regression guard (§11.4.135):** 5 new Vitest specs in dashboard.component.spec.ts (instant button processing indicator (isBusy)) asserting busy true in-flight / false

after resolve AND after error, for each of the three actions. Full dashboard suite **103/103 GREEN. Verified (§11.4.108):** the deployed bundle contains btnBusyPulse / btn-spinner (served == built).

2026-06-16 — search broken / “crashing a lot”: multi-word query encoding storm, private-tracker auth, deploy pipeline, healthz, kickass

Operator-reported: “search is broken / crashing a lot” and “rutracker/nnmclub never authenticate”. Root-caused live on the distributed-boot production stack (nezha.local, podman) — §11.4.107 / §11.4.123 anti-bluff. Full QA evidence: docs/qa/search-fix-verify-20260616/ (results.md + html/pdf/docx). Definitive end-to-end proof: full-fleet the matrix on nezha returned **2600 results, 23/29 trackers contributing, encoding-crashed: NONE, all four private trackers authenticated** (rutracker 50 / nnmclub 50 / kinozal 50 / iptorrents 49).

48. Multi-word query URL-encoding crash across 17 nova3 plugins

Severity: HIGH (multi-word search silently lost ~17 plugins; the user-visible “search broken / crashing”). **Type:** Bug. **Root cause (FACT, captured live):** ~17 nova3 engine plugins interpolated the RAW query string into a request URL. A literal space (any multi-word query) crashed urllib with URL can't contain control characters. Single-word queries worked; multi-word queries silently dropped each affected plugin from the results — so the fleet returned a fraction of its real coverage and looked “broken / crashing a lot”. **Fix:** per-plugin percent-encoding of the query before URL interpolation, across 7 maintained engines (commit dbd3858) + 10 adopted/orphan engines incl. limetorrents (commit da7d709, which also adopted 9 orphan engines into plugins/ per §11.4.124). **Affected:** the 17 affected plugins/*.py nova3 engines (e.g. glotorrents, torrentdownload, torrentproject, snowfl, limetorrents, torrentscsv, bitsearch, torrentgalaxy, nyaa, rockbox, ...). **Regression guard (§11.4.115/§11.4.135/§11.4.85):** multiword URL-oracle unit tests (16-plugin capture + snowfl focused + well-behaved-no-regression) hardened for py3.13 (f991034); stress+chaos coverage for the multi-word fix + classifier (7e9cab5). **Verified live (§11.4.107):** scoped multi-word the matrix of the adopted plugins → total **592** (glotorrents 173, torrentdownload 249, torrentproject 68, snowfl 40, limetorrents 36, torrentscsv 25, rockbox 1), and the full-fleet run returned **2600** — STILL bad_query_encoding/crashed: NONE. Earliest definitive proof: nezha result counts confirm a real torrent like “Матрица / The Matrix” surfaced, 0 plugins crashed.

49. plugin_crashed mislabel — un-encoded multi-word query is a code defect, not a tracker crash

Severity: MEDIUM (honesty/diagnostic — §11.4.6: a self-inflicted encoding bug was being reported as if the remote tracker had crashed). **Type:** Bug. **Root cause (FACT):** the orchestrator classified the urllib control-character failure from an un-encoded multi-word query as plugin_crashed, conflating “we sent a malformed URL” with “the tracker is down” — the exact §11.4.6 guessing pattern. **Fix:** added a distinct plugin_bad_query_encoding failure class so the un-encoded-query defect is reported honestly as a code defect, not a tracker crash (commit 33d90f2).

Affected: download-proxy/src/merge_service/ search orchestration / per-plugin failure classification. **Regression guard (§11.4.115/§1.1):** classifier unit test with the §1.1 negation proof (un-encoded query → plugin_bad_query_encoding, NOT plugin_crashed).

50. Private-tracker cookie auth — rutracker (and nnmclub) never authenticated

Severity: HIGH (rutracker/nnmclub failed every query — no_cookie/CAPTCHA on rutracker, Cloudflare Turnstile on nnmclub). **Type:** Bug. **Root cause (FACT):** rutracker's login endpoint is CAPTCHA-walled and nnmclub's is behind Cloudflare Turnstile, so the username/password login path could never authenticate; there was no path to inject the operator's already-valid browser session cookies for rutracker. **Fix:** RUTRACKER_COOKIES injection that bypasses the CAPTCHA-walled login (commit 2fc29fc), complementing the existing NNMCLUB_COOKIES; cookies are extracted from the operator's browser export by scripts/extract-tracker-cookies.sh (only the tracker's own-domain cookies, §11.4.10). **Affected:** plugins/rutracker.py (cookie injection), scripts/extract-tracker-cookies.sh, container env wiring. **Regression guard (§11.4.115):** rutracker-cookie unit test with §1.1 negation. **Verified live (§11.4.107/§11.4.69):** on nezha, rutracker the matrix → **50** real results (sample title “Матрица / The Matrix (Энди/Ларри Вачовски)”); nnmclub → **50** (“The Matrix OST (1999)”, “Matrix Resurrections (2021)”); container env confirmed carrying both cookie vars (phpbb2mysql_4_sid=1, bb_session=1).

51. /auth/status did not reflect injected cookies (red chips despite working searches)

Severity: MEDIUM (user-visible — the dashboard showed red/unauthenticated chips for rutracker/nnmclub even though their searches worked). **Type:** Bug. **Root cause (FACT):** /auth/status only read _tracker_sessions, which is populated AFTER the first search, so it ignored the *_COOKIES env-injected sessions and reported them unauthenticated on a fresh stack. **Fix:** /auth/status now also reflects the *_COOKIES env so the dashboard shows cookie-authenticated trackers green before the first search (commit 9c2f8dc). **Affected:** download-proxy/src/api/routes.py (auth/status handler). **Regression guard (§11.4.115/§1.1):** auth-status cookie-reflection unit test with negation proof.

52. /api/v1/healthz returned the SPA HTML, not JSON (swallowed by the catch-all)

Severity: MEDIUM (health/observability — the health endpoint returned the SPA index instead of a JSON health body, breaking machine health checks). **Type:** Bug. **Root cause (FACT):** the SPA catch-all route matched /api/v1/healthz before the health handler, so the endpoint served HTML. **Fix:** dedicated JSON /api/v1/healthz endpoint registered ahead of the SPA catch-all (commit 137d7ff). **Affected:** download-proxy/src/api/routes.py (route ordering + healthz handler). **Regression guard (§11.4.43):** healthz JSON-contract unit test (asserts JSON body, not HTML).

53. scripts/deploy-remote.sh — 4 silent-failure bugs kept fixes from landing on the remote

Severity: HIGH (§11.4.108 — fixes appeared committed but the deploy pipeline never delivered them to the running containers). **Type:** Bug. **Root cause (FACT):** four independent silent failures in the deploy pipeline: (a) an inline YAML comment was left in a parsed path; (b) under set -e, rsync aborted on container-owned config/; (c) py_compile wrote __pycache__ into a container-owned dir → false “Syntax: Invalid” → exit 1; (d) install-plugin targeted only the qbittorrent container, not the qbittorrent-proxy container that actually runs the engine subprocess. **Fix:** (a) strip inline YAML comments from the parsed path + exclude container-owned engines (d5b58cb); (b) exclude container-owned config/ + tolerate rsync exit 23/24 (9e059d3); (c) syntax-check via compile() not py_compile so nothing is written to the container dir (42cdb02); (d) install engines into qbittorrent-proxy too, not just qbittorrent (e6b9f8f). **Affected:** scripts/deploy-remote.sh, install-plugin.sh. **Regression guard (§11.4.108):** pipeline re-run completes [1/5]→[5/5] cleanly (“All plugins installed and valid!”, 0 “Syntax: Invalid”). **Verified live (§11.4.108):** re-run AFTER a clean pipeline deploy (engines installed by the pipeline, NOT a manual cp) — scoped multi-word the matrix total 573; engine fix confirmed present in qbittorrent-proxy (glotorrents fix marker count 1) — installed by the pipeline, not hand-copied.

54. kickass plugin — 403 is structurally blocked (Cloudflare / JS challenge), classified Won't-fix (§11.4.112)

Severity: LOW (one community plugin cannot be made to work). **Type:** Bug → Won't-fix (structurally-impossible). **Root cause (FACT, researched):** kickass returns HTTP 403 behind a Cloudflare / JavaScript challenge that a headless nova3 plugin cannot solve; there is no server-side cookie-injection or header path that bypasses it. **Resolution (§11.4.112):** classified Won't-fix: structurally-impossible with the cited evidence documented in docs/research/ (commit f3e7a4f). No further attempts unless the platform constraint changes (§11.4.34/ §11.4.7). **Affected:** plugins/kickass.py, docs/research/.

55. UTF-8 / Cyrillic query crash in 14 plugins

Severity: HIGH. **Type:** Bug. **Status:** FIXED (ae387b2) — verified live on nezha. **Root cause (FACT):** sibling defect class to #48 — 14 nova3 plugins crashed on a UTF-8 / Cyrillic (non-ASCII) query with 'ascii' codec can't encode characters (captured stderr from the §11.4.118 discovery sweep). #48's fix only .replace(" ", ...) -ed literal spaces; the raw non-ASCII bytes still reached urllib and crashed. Surfaced during the #48 discovery extend-pass (§11.4.146 / §11.4.118). **Fix (ae387b2):** per-plugin UTF-8-safe query encoding (all unquote* first to avoid double-encoding the nova2 caller): quote_plus(unquote_plus(q)) for ?key= params (bitsearch, glotorrents, linuxtracker, nyaa, pirateiro, rockbox, torrentdownload, torrentscsv, torrentproject, tokyotoshokan + re_escape for its pagination regex); quote(..., safe='') for path segments (torrentgalaxy, snowfl); "-".join(quote(word, safe='')...) for dash-slugs (torlock, yourbittorrent). §11.4.6: yts was NOT affected (already uses urlencode()) — left unchanged → 14, not 15. **Affected:** the 14 plugins/*.py above. **Regression guard (§11.4.115/ §11.4.135):** tests/unit/test_plugin_unicode_query_encoding.py (RED 14 failed → GREEN 97 passed incl. 54 existing multiword; §1.1: strip nyaa encoding

→ FAIL → restore). **Verified:** live nezha Cyrillic search “Война и мир” → **290 results, CRASHED/encoding-err: NONE** (bitsearch 80, torrentgalaxy 198, torrentscsv 11, torrentdownload 1 success; rest empty-no-error / deadline_timeout, zero crashes). Evidence: docs/qa/search-fix-verify-20260616/.